

Resumen de las principales formas de procesar ficheros en Java

1) Introducción

Desde el punto de vista software, todo en informática son números. La interpretación que los programas hacen de estos números es la que permite clasificar los datos en tipos. Cuando estos datos se agrupan y organizan adecuadamente se transforman en información. Y para que esta información pueda ser persistente se almacena en ficheros.

2) Ficheros de texto

Son aquellos ficheros en los que los números que contienen se deben interpretar en base a un código. En el caso concreto del lenguaje de programación Java, el código empleado es Unicode. Si analizamos con alguna herramienta que nos proporcione acceso a los valores numéricos que contiene un fichero, podremos observar lo que realmente contiene. Por ejemplo, en el proyecto Java adjunto a este documento hay un fichero de texto llamado java.txt que contiene en su interior el texto "JAVA 2025 3,1415". Si abrimos este fichero con el editor hexadecimal online: <https://hexed.it/> podremos ver que lo que contiene realmente el fichero es la secuencia numérica (en hexadecimal): 4A 41 56 41 20 32 30 32 35 20 33 2C 31 34 31 35. Cuando abrimos este fichero con un editor de texto, el editor interpreta cada número como código Unicode y por lo tanto nos muestra el texto "JAVA 2025 3,1415".

2.1) Lectura carácter a carácter

Cuando deseamos procesar en Java un fichero de texto carácter a carácter el mecanismo más sencillo es utilizar la clase FileReader. Una vez abierto el fichero, el procesamiento del mismo se realiza con un bucle while cuya condición incluye la lectura y la verificación de lo leído. Como se puede observar en el ejemplo, en realidad se leen enteros (int) pero luego se procesan como si fuesen char. La finalización del fichero se detecta cuando la lectura es -1 pues ese valor no corresponde con ningún código Unicode y es la forma de indentificar que hemos alcanzado el final del fichero (end of file - EOF).

```
FileReader entrada = new FileReader(fileName);
int c;
while ((c=entrada.read())!=-1)
{
    letra = (char)c;
    System.out.println("Leo la letra: "+letra);
}
entrada.close();
```

2.2) Escritura carácter a carácter

```
FileWriter salida = new FileWriter(fileName);
for (int i=0;i<texto.length();i++)
    salida.write((int)texto.charAt(i)); // Letra a letra

salida.write(texto); // Admite también el string
salida.close();
```

2.3) Lectura línea a línea

Si optamos por realizar lecturas de líneas completas el mecanismo más sencillo es utilizar la clase `BufferedReader`. Una vez abierto el fichero, el procesamiento del mismo se realiza con un bucle `while` cuya condición incluye la lectura y la verificación de lo leído. Como se puede observar en el ejemplo se leen líneas completas. La finalización del fichero se detecta cuando la lectura es `null` (ausencia de cadena).

```
FileReader fr = new FileReader(fileName);
BufferedReader entrada = new BufferedReader(fr);
String linea;
while ((linea=entrada.readLine())!=null)
{
    System.out.println("Leo la línea: "+linea);
}
entrada.close();
fr.close();
```

Este mecanismo suele ir asociado al procesamiento de ficheros que contienen palabras. Para ello, cada vez que leemos una línea en el bucle `while`, dividimos la misma en “tokens” utilizando el método `split` con una expresión regular que nos indique el/los caracteres separadores y se procesa el vector de palabras que retorna `split` con un bucle `for`

```
FileReader fr = new FileReader(fileName);
BufferedReader entrada = new BufferedReader(fr);
String linea;
String [] tokens;
while ((linea=entrada.readLine())!=null)
{
    System.out.println("Leo la línea: "+linea);
    tokens = linea.split(separador);
    for (int i=0;i<tokens.length;i++)
        System.out.println("Token: "+tokens[i]);
}
entrada.close();
fr.close();
```

2.4) Escritura línea a línea

En este caso la escritura se podría realizar carácter a carácter o línea a línea, siendo lo más frecuente hacerlo a nivel de string (líneas o tokens).

```
FileWriter fw = new FileWriter(fileName);
BufferedWriter salida = new BufferedWriter(fw);
salida.write(texto);
salida.close();
fw.close();
```

2.5) Lectura de tipos simples

Por último, cuando deseamos leer un fichero de texto por líneas completas, palabras, números enteros y/o números reales, podemos usar un `Scanner`. En ese caso, la condición del bucle `while` siempre será `hasNextLine()` y, o bien sabemos con certeza cómo está organizado el fichero de texto y vamos leyendo lo que corresponde en cada momento (ahora un entero, luego un string, etc), o bien comprobamos antes de leer si lo que queremos leer está disponible.

```

FileReader fr = new FileReader(fileName);
Scanner entrada = new Scanner(fr);
while (entrada.hasNextLine())
{
    if (entrada.hasNextInt())
    {
        entero= entrada.nextInt();
        System.out.println("Leo un número entero:"+entero);
    }
    else
        if (entrada.hasNextDouble())
        {
            real= entrada.nextDouble();
            System.out.println("Leo un número real:"+real);
        }
        else
            if (entrada.hasNext())
            {
                palabra=entrada.next();
                System.out.println("Leo una palabra: "+palabra);
            }
            else
                entrada.nextLine();
}
entrada.close();
fr.close();

```

2.6) Escritura de tipos simples

La escritura en este caso es muy sencilla pues podemos hacerlo exactamente igual que imprimos en pantalla, por lo que tenemos a nuestra disposición métodos como print, println, printf, etc.

```

FileWriter fw = new FileWriter(fileName);
PrintWriter salida = new PrintWriter(fw);

salida.println ("Texto "+palabra);
salida.println ("Entero "+entero+" ");
salida.println ("Real "+real+" ");
salida.close();
fw.close();

```

3) Resumen ficheros de texto

FileReader FileWriter	<pre> int c; while ((c=entrada.read())!=-1) { letra = (char)c; System.out.println("Leo la letra: "+letra); } </pre>
BufferedReader BufferedWriter	<pre> while ((linea=entrada.readLine())!=null) { System.out.println("Leo la línea: "+linea); } </pre>
Scanner PrintWriter	<pre> while (entrada.hasNextLine()) entrada.nextInt(); entrada.nextDouble(); etc... </pre>

4) Ficheros binarios

En un fichero binario, los datos se almacenan de acuerdo a la forma de codificarlos que utilice el lenguaje. En Java se utilizan código Unicode para los char, binario para los int, punto flotante para los reales. Además, en función del subtipo, se utilizan diferente cantidad de bytes (1 para char, 4 para int, 8 para double, etc.)

El fichero java.dat contiene el mismo contenido que el fichero java.txt pero en binario. Si se abre con el editor hexadecimal, se puede observar que los códigos de los textos coinciden, aunque al tratarse de un String se incorpora delante la longitud codificada en binario de la cadena. Esto último se hace para que al leerlo se pueda saber cuántos bytes ocupa el String. En el caso de los números, la forma de almacenar el valor 2025 en binario no tiene nada que ver con la forma de almacenar ese valor en texto. En los ficheros de texto se guardaban los códigos Unicode de cada dígito, sin embargo en el fichero binario se convierte el valor a binario y se almacena como la secuencia de bytes (cuatro por ser entero). Lo mismo ocurre con el real (8 bytes por ser double) aunque la forma de representar es diferente (mantisa y exponente).

El fichero binario tiene esta secuencia en su interior: 00 04 4A 41 56 41 00 00 07 E9 40 09 21 CA C0 83 12 6F. Los dos primeros bytes 00 y 04 representan la longitud de la palabra que viene a continuación (4 letras). Los bytes 4A 41 56 41 es la codificación de la palabra JAVA en Unicode (igual que en el fichero de texto). Los bytes 00 00 07 E9 representan la codificación del número 2025. Y los bytes 40 09 21 CA C0 83 12 6F representan al 3.1415.

Para los ficheros binarios hemos visto cuatro métodos, pero en realidad son dos los relevantes. Como clase base para todos ellos usamos FileInputStream y FileOutputStream.

4.7) Lectura de tipos simples

Los ficheros binarios no tienen mecanismos “elegantes” como ocurre en otros lenguajes para detectar el final del fichero. Por ello se recurre a la detección de excepciones que no es más que leer hasta que se produzca un error. Los métodos para leer los tipos más comunes son readUTF() para cadenas, readInt ()para enteros, readDouble() para doubles, etc...

```
FileInputStream fis = new FileInputStream(fileName);
DataInputStream entrada = new DataInputStream(fis);
while (true)
{
    try
    {
        String cadena = entrada.readUTF();
        int entero = entrada.readInt();
        double real = entrada.readDouble();
        System.out.println("Leo cadena "+cadena);
        System.out.println("Leo entero "+entero);
        System.out.println("Leo real "+real);
    }
    catch (EOFException e)
    {
        System.out.println("Fin del fichero alcanzado (EOF)");
        break; // Para abandonar el bucle
    }
}
entrada.close();
fis.close();
```

4.8) Escritura de tipos simples

En este caso resulta tan sencillo como invocar al método adecuado en virtud del tipo de dato que queramos almacenar: writeUTF() para cadenas, writeInt() para enteros, etc...

```
FileOutputStream fos = new FileOutputStream(fileName);
DataOutputStream salida = new DataOutputStream(fos);
salida.writeUTF(texto);
salida.writeInt(entero);
salida.writeDouble(real);
salida.close();
fos.close();
```

4.9) Lectura de objetos

En el caso de los objetos, la premisa que deben cumplir las clases a las que pertenecen los objetos que queremos leer o escribir es que implementen la interface Serializable.

```
FileInputStream fis = new FileInputStream(fileName);
ObjectInputStream entrada = new ObjectInputStream(fis);
while (true)
{
    try
    {
        Alumno alumno = (Alumno) entrada.readObject();
        System.out.println("Leo "+alumno);
    }
    catch (EOFException e)
    {
        System.out.println("Alcanzado EOF");
        break;
    } catch (ClassNotFoundException e)
    {
        e.printStackTrace();
    }
}
entrada.close();
fis.close();
```

4.10) Escritura de objetos

Sucede de nuevo lo mismo, almacenar un objeto es tan sencillo como invocar al método writeObject(). Además se pueden almacenar objetos individuales o bien colecciones completas. Eso sí, si se almacenan objetos individuales hay que leer los objetos de forma individual y si se almacenan colecciones completas hay que leer la colección completa.

```
FileOutputStream fos = new FileOutputStream(fileName);
ObjectOutputStream salida = new ObjectOutputStream(fos);
salida.writeObject(alumno); salida.flush();
salida.close();
fos.close();
```

5) Resumen ficheros binarios

FileInputStream FileOutputStream	Es la clase base para los binarios. La lectura siempre es un bucle while (true) y se controla la excepción EOFException.
DataInputStream DataOutputStream	readUTF(), readInt(), readDouble() y writeUTF(), writeInt(), writeDouble()
ObjectInputStream ObjectOutputStream	readObject(), writeObject() y flush()